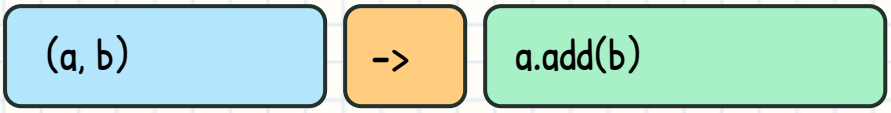
Lambdas in depth · my notes L03

a lambda = a block of code you can pass around, with strict rules

1 · Anatomy



parameters
( ) optional for 1 inferred param

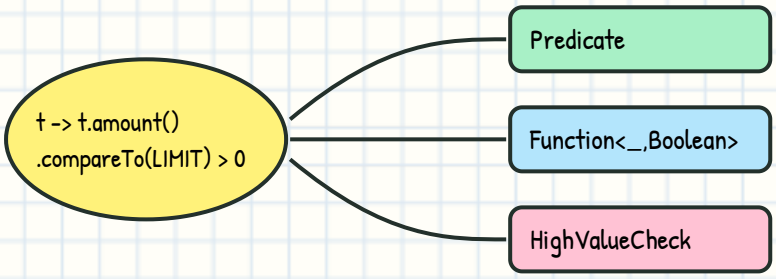
arrow

body = expression
its value is returned

block body

```
a -> {
    ...;
    return x;  ← needed!
}
```

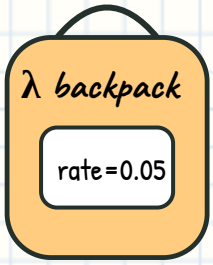
2 · Target typing = one shape-shifter, many sockets



The **context** decides: assignment, method argument, return, `? :` , cast.

No context? `Object o = () -> "x";`
→ Object is not a functional interface

3 · Capture = the lambda packs a COPY in its backpack



✓ `rate` set once → effectively final → 80.00 × 5% = 4.00

✗ `ids.forEach(id -> count++)`
must be final or effectively final

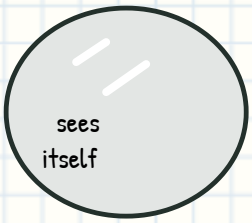
for-each: new f each turn → [SQ318, SQ12, TR808] ✓
classic for: copy `int pos = i + 1;` first

Why? the lambda may run later or on another thread, so its copy must never go stale. Fields are read live.

4 · this: window vs mirror

sees the	outer
object	service

lambda = window



anonymous = mirror

lambda → `payment-service`

anonymous → `anonymous object`

No new scope in a lambda: `String id; … id -> …`

already defined!

5 · Counter hacks = smell

`AtomicInteger` , `int[] {0}` in forEach: compiles, gives 45… but hides mutable state, and the array races in parallel.

✓ `filter(…).count()` → `45`

6 · var [11]

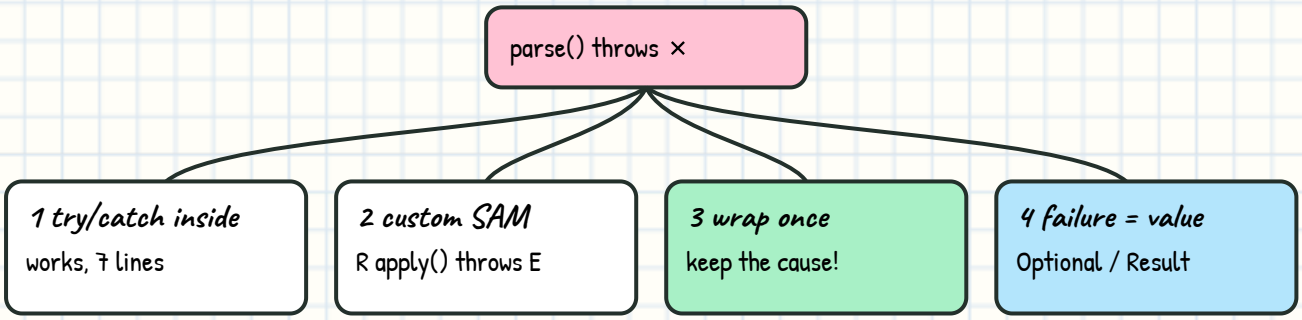
`(final var t) -> …`

`(@Audited var t) -> …`

all var or none!

`(a, _) -> a` = preview in 21, final 22

7 · Checked exceptions: four roads



8 · Under the hood

anonymous → `Outer$1.class` file, new object each time

lambda → `invokedynamic` → hidden class [15], no file

never `==` or lock on a lambda

★ Remember

- expression returns, block needs return
- effectively final = never re-assigned
- > 3 lines → named method
- next → L04 method references